

MAJOR PROJECT · DEPARTMENT OF COMPUTER APPLICATIONS (BCA)

PlaceMate

A Full-Stack Placement Cell Management System
with Server-Side Policy Enforcement

Presented by

Tushar Kumar

Department of Computer Applications (BCA)
Maharaja Surajmal Institute, C-4 Janakpuri,
New Delhi – 110058
Affiliated to Guru Gobind Singh Indraprastha
University, Delhi

Flask 3

React 19

PostgreSQL

Vite + Tailwind 4

Recharts

Vercel + Neon

September 2026

What this presentation covers

- | | | |
|----|--------------------------------------|--|
| 01 | Problem & motivation | Why placement season breaks down on spreadsheets |
| 02 | Objectives | The four contributions this project makes |
| 03 | System design | Architecture, data model, eligibility engine, offer policy |
| 04 | Student portal | Every screen a student uses, end to end |
| 05 | Teacher / admin portal | Every screen the placement cell uses, end to end |
| 06 | Evaluation | 23 functional assertions, payload and query measurements |
| 07 | Limitations & future work | Honest gaps and the roadmap that follows |

Placement season is a small operations centre

Hundreds of students, dozens of drives, rules that do not bend — coordinated by hand.

The load

- Several hundred final-year students
- A few dozen visiting companies
- A rolling calendar of recruitment drives
- Minimum CGPA, department limits, deadlines

What goes wrong

- Spreadsheets drift out of date as staff edit in turns
- Nobody can say later why a student was rejected
- Students hear about drives days late, apply by email
- The office re-enters every application by hand

What is missing

- No single place that decides eligibility
- No record of who changed a decision, or when
- No protection against a student taking a worse offer
- No live picture of the season for the cell

OBJECTIVES

Four contributions

Each one is implemented and verified, not proposed.

01

One eligibility engine

Eligibility rules live in a single server-side function used by four call sites, so the student view and the admin view cannot disagree.

02

Offer ladder policy

An automatically maintained placement-record table stops a placed student from sitting for a lower-paying drive.

03

Append-only audit trail

Every status transition appends a row: stage, note, acting user, timestamp. Decisions can be replayed.

04

Measured performance

Code splitting plus joined queries cut the first payload by more than half and removed per-row database work.

Stack and deployment

A conventional three-tier web application, deliberately.

Frontend

React 19 · Vite · Tailwind CSS 4 · React Router · Recharts

Role-guarded routes, lazy-loaded dashboard, compositor-only animation

Backend

Python 3 · Flask 3 application factory · Flask-SQLAlchemy · Flask-CORS

Three blueprints: auth, admin, portal. Role-checking decorator on every route

Data

PostgreSQL via psycopg · SQLAlchemy ORM · pandas aggregation

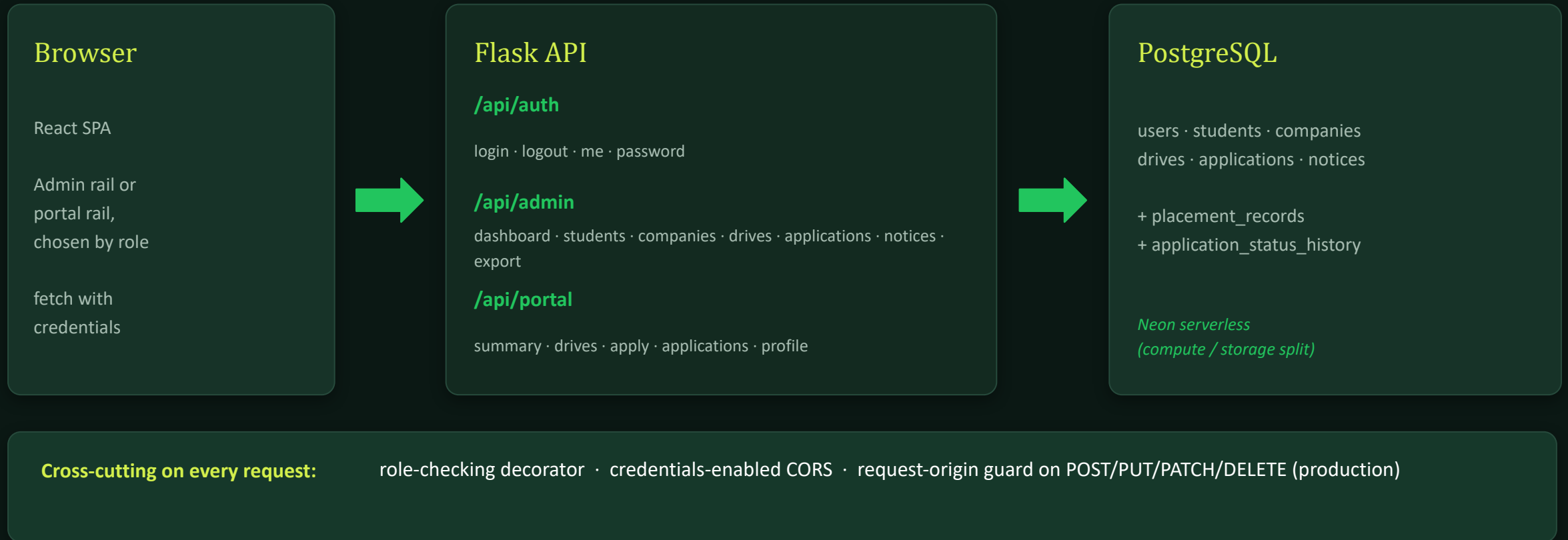
Neon serverless Postgres in production; unique constraints enforce core rules

Delivery

GitHub · Vercel (frontend + API as separate services) · Neon

Session cookie, CORS, and an origin guard handle the split deployment

Request path through the system



Two portals, two audiences

One codebase, one API, two role-guarded experiences.

Student portal

/portal

- See only the drives they are actually eligible for
- Apply in one click, or read exactly why they cannot
- Track each application and its stage history
- Keep phone and skills current, change password
- See upcoming deadlines and notices on the home page

Teacher / placement cell (admin)

/dashboard

- Maintain students, companies, drives and notices
- Import a cohort by CSV, with per-row error reporting
- Move candidates between four stages; every move is logged
- Read a descriptive dashboard of the season
- Override an offer's actual CTC; export major tables as CSV

Six core tables, two that carry the policy

Table	Holds	Key columns	Role in the design
users	Login identities	email, password_hash, role	Role drives portal routing
students	Student records	roll_no, department, program, cgpa, skills	Eligibility inputs + profile
companies	Recruiters	name, sector, website	Parent of drives
drives	Recruitment drives	role, package_lpa, min_cgpa, eligible_departments, eligible_programs, application_deadline, is_active	Eligibility inputs
applications	Student × drive link	student_id, drive_id, status, applied_at	Unique (student, drive) stops duplicates
notices	Announcements	title, body, created_at	Student and admin notice board
placement_records	Actual offers	student_id, drive_id, package_lpa, offered_on	Backs the offer ladder
application_status_history	Audit trail	application_id, status, note, changed_by, created_at	Append-only; never updated

One eligibility function, four call sites

It returns an allow flag and, when the answer is no, the reason string the student will read.

The checks, in order

- 1 Application deadline has not passed
- 2 The drive is active, not closed
- 3 Student's program is in eligible_programs
- 4 Student's department is in eligible_departments
- 5 CGPA clears the drive's minimum

The offer ladder is a separate check, applied only at apply time

Four call sites, one source of truth

Student drive listing

Decides Apply vs Not eligible, and supplies the reason

Apply endpoint

Re-checks on the server; an ineligible POST is refused

Admin application badge

Same verdict shown to the placement cell

Eligible-but-not-applied list

How the cell sees who it should chase

The offer ladder

A placed student may only apply to drives paying at least their best offer + 2.0 LPA.

1

Marked selected

The admin moves an application to selected.

2

Offer recorded

A placement record is inserted:
student, drive, date, package seeded
from the drive.

3

CTC overridden

The cell replaces the advertised
figure with the actual CTC.

4

Ladder enforced

Apply is refused unless package $\geq$
best offer + 2.0 LPA.

Worked example

Aarav holds an offer at 6.0 LPA, so the threshold is 8.0 LPA.

Drive at 9.0 LPA → allowed

Drive at 7.0 LPA → refused, with reason

Drives with no published package stay open — the rule cannot be evaluated against them. Dashboard package figures read actual offers when they exist, and fall back to advertised figures otherwise.

Every decision leaves a trace

application_status_history is append-only: rows are added, never updated, never deleted.

What a row records

application_id	which application moved
status	the new stage: applied, shortlisted, selected, rejected
note	optional context the cell wants on the record
changed_by	the user who made the move
created_at	timestamp, added automatically

The initial application is itself the first row, written in the same transaction as the application.

Where it surfaces

● Admin application detail

A full timeline of the application's life

● Student applications list

Compact stage chips under each application

● Season review

The cell can reconstruct when any decision was made

● Cascade delete

Rows are removed only when the whole application is

Sessions in a split deployment

Because the interface and the API run on different subdomains, the cookie choice forces a guard.

Signed session cookie

Flask's client-side session, signed with SECRET_KEY. Production refuses to boot without a real key, so the development fallback cannot leak into a deployment.

SameSite=None + Secure

Required for the cross-site cookie the split deployment needs — and it means the browser will also attach the cookie to cross-site POSTs.

Request-origin guard

Before any POST, PUT, PATCH or DELETE, the Origin (or Referer) host must match the API's own host or the configured frontend origin. A browser-sent cross-site POST gets 403.

Header-less clients allowed

Non-browser clients send neither header; they are allowed. The guard runs only in production, so the Vite dev proxy is unaffected.

PART ONE

Student portal

Four screens, reachable from a mobile tab bar as well as the desktop rail.
Every drive a student sees already carries its own eligibility verdict.

Home: the season at a glance

Route /portal — greeting, placement status, four counters, deadlines, notices, quick actions.

APPLIED

4

SHORTLISTED

2

SELECTED

1

REJECTED

1

Upcoming deadlines

Nimbus Analytics · Data Analyst

8.5 LPA

12 Oct

Vertex Systems · QA Engineer

6.0 LPA

18 Oct

Helios Retail · Business Analyst

Package TBD

24 Oct

Latest notices & quick actions

- Apply to drives — browse what you qualify for
- Track applications — every stage so far
- Update profile — keep phone and skills current
- Read notices — posted by the placement cell

Drives: eligibility shown up front

Route /portal/drives — each card carries the verdict, so the student never guesses.

Eligible

Nimbus Analytics

Data Analyst · Full time

8.5

7.0

—

LPA

MIN CGPA

STATUS

APPLY NOW

CGPA 8.1 clears the 7.0 minimum

Already applied

Vertex Systems

Data Analyst · Full time

6.0

6.0

—

LPA

MIN CGPA

STATUS

APPLIED · SHORTLISTED

Duplicate applies are refused by a unique constraint

Not eligible

Orion Labs

Data Analyst · Full time

12.0

8.5

—

LPA

MIN CGPA

STATUS

NEEDS CGPA ≥ 8.5

The reason string comes from the engine

PlaceMate · Tushar Kumar · BCA

15

Applications: stage and history

Route /portal/applications — each row shows package, drive date and the stage chips.

Nimbus Analytics · Data Analyst

Analyst drive 2026

8.5 LPA

12 Oct

02 Sep

Shortlisted

Package

Drive date

Applied

Status history from the audit table

Applied

02 Sep 2026

Application created

Shortlisted

05 Sep 2026

Cleared the aptitude round

Shortlisted

07 Sep 2026

Moved to the technical panel

Profile and account

Route /portal/profile — the student maintains the record the placement cell relies on.

Personal details

NAME	EMAIL	PROGRAM
Aarav Sharma	aarav@example.com	B.Tech
DEPARTMENT	CGPA	
CSE	8.10	

Editable by the student

- Phone number
- Skills, as a comma-separated list
- Change own password (current + new)

How a student gets an account

No self-registration

Identities are created by the placement cell, not by the public.

Random temp password

A generated value shown to the admin at creation or reset. Not derived from roll numbers, and not a fixed default.

First-login change

The student changes it from this screen via the password endpoint.

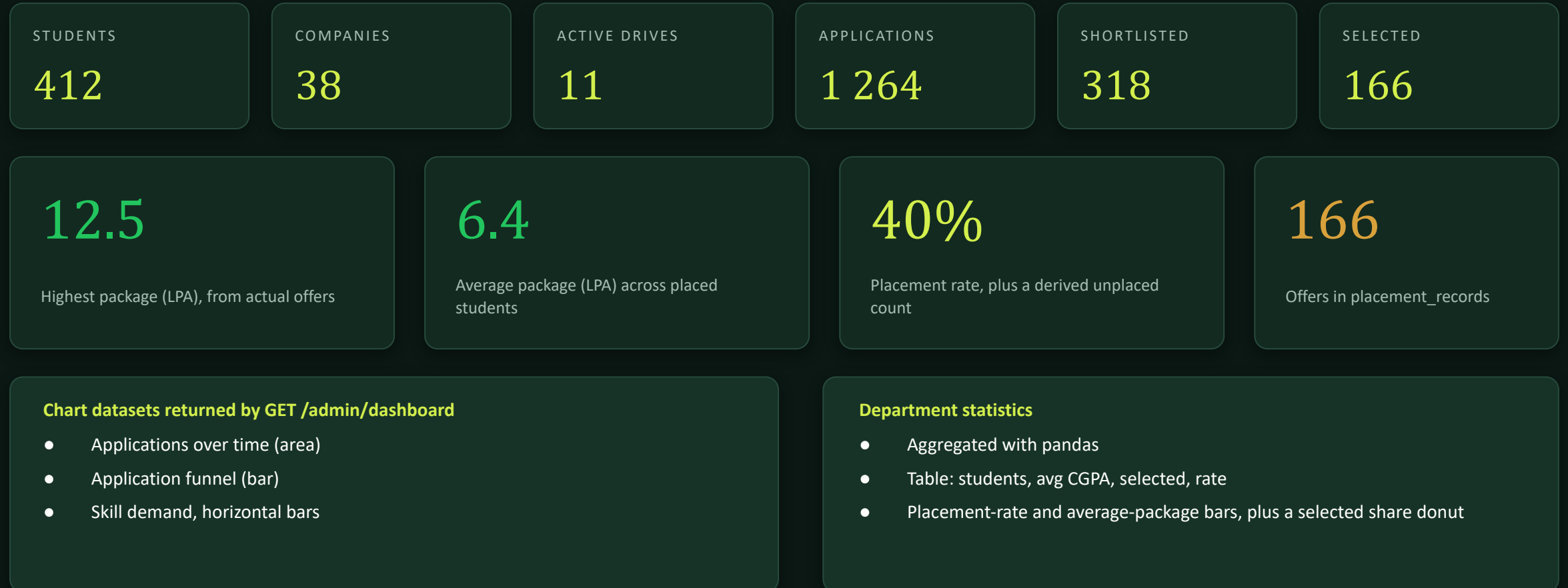
PART TWO

Teacher / placement-cell portal

The operations console. Six rail sections: Home, Students, Companies, Drives, Applications and Notices — plus CSV import and export on every major table.

Home: the whole season on one screen

KPIs, an application trend, a funnel, skill demand, and department statistics.



Students: the cohort record

Full CRUD, one-time password provisioning, CSV import with per-row errors, CSV export.

What the screen does

- Search by name, roll number or email
- Filter by department and by program
- Add a student individually, or import a CSV
- Reset a login password on demand
- Open a student to see their applications and offers
- Export the filtered table as CSV

CSV import, done carefully

- Every column parsed as text, so roll and phone numbers survive
- File capped at 5 000 rows
- Bad rows reported with their reason, the rest still imported
- A downloadable template is linked on the page

Two details worth defending

Each created student gets a random one-time password, returned once — shown on creation or on an explicit reset. The bulk import returns the same list for the admin to distribute.

The export prefixes any cell starting with = + - or @ with an apostrophe, so a CSV opened in Excel or Sheets cannot execute a formula.

Companies and drives

A drive is the unit of work: one company, one role, one set of rules, one deadline.

Companies

- Name, sector, website, contact details
- Search and filter the recruiter list
- Detail view lists every drive for that company
- Deleting a company cascades to its drives and their applications

Drives

- Role, package, minimum CGPA, drive date
- Eligible departments and eligible programs, per drive
- Application deadline, and an active/closed toggle
- Targets view: eligible students who have not applied yet
- Delete a drive and its applications in one action

The targets view is the quiet win:

for any drive, the cell can list the students who are eligible but have not applied — the actual follow-up list, computed by the same eligibility function that gates the student's Apply button. Placed students are excluded by the offer ladder.

Applications: decisions and history

Every status move appends an audit row and, where relevant, touches the offer table.

● Applied

creates the first history row

● Shortlisted

may add a note

● Selected

creates a placement record

● Rejected

deletes any placement record

List and detail

- Filter by status, department and company
- Every row shows the live eligibility badge and its reason
- Detail page shows the student, the drive, and the full timeline
- Notes can be attached to a move, and appear in the history

Why the badge matters here

The placement cell sees the same verdict the student saw. If a candidate is ineligible, the officer sees the identical reason string — not a different message produced by a second implementation of the rules.

When an application is marked selected, a placement record is created and the offer appears on the student's detail page. Walking the decision back removes it, so offers and statuses stay consistent.

Student detail: applications and offers

The one screen where the cell corrects the package a student actually received.

What the officer sees

- Profile: roll number, program, department, CGPA, phone, skills
- Every application with its current stage and eligibility reason
- The placement offers attached to this student
- The offered package, editable in place
- A delete path that cascades to their applications, history and offers

Offer editor

PUT /api/admin/students/:id/offers/:rid

Nimbus Analytics

Advertised

8.5 LPA

Nimbus Analytics

Actual CTC

9.2 LPA

The recorded CTC is what the ladder measures against, and what the dashboard's highest and average package figures use. Drives with no advertised package stay exempt until a figure is recorded.

Notices, exports and account tools

The communication surface, plus the escape hatches offices genuinely need.



Notices board

- Post an announcement with a title and body
- Visible to students on their home screen and notices list
- Delete a notice when it is stale
- The natural home for drive reminders and policy changes



CSV export

- Students, companies, drives and applications
- Honours the filters currently applied
- Formula-prefixed cells neutralised for Excel and Sheets
- Lets the office keep its own reporting workflow



Accounts & self-service

- Change own password from any portal
- Reset a student's password, returning a fresh one-time value
- Admin account auto-seeded on first backend start
- Student logins auto-provisioned with the student record

Feature matrix: who can do what

Role-checking is enforced server-side on every route, not by hiding buttons.

Capability	Student	Teacher / cell
Sign in, change own password	Yes	Yes
Manage students, companies, drives, notices	—	Yes
Browse drives	Only those passing eligibility	All, including inactive
Apply to a drive	One click, server-validated	—
See the ineligibility reason	Yes, on the card	Yes, as a badge
Track application stage history	Own applications	Every application
Read notices	Yes	Posts and deletes them
Record / override an offer's CTC	—	Yes
Offer ladder enforced	Blocks their apply	Excluded from targets view
Import a cohort / export CSV	—	Yes
Season analytics dashboard	Personal counters only	Full dashboard

Functional verification

A scripted integration suite drove the running application over real HTTP against a scratch database.

23

REST-level assertions in the development integration suite, passing before and after the performance work

2×

The suite was run twice — the second run is the evidence that optimisation changed cost, not behaviour

403

Returned to a foreign origin, while the configured origin and header-less clients pass

Scratch DB

Created for the run and dropped afterwards — no demo data was touched.

What the assertions covered

- Admin login, then creating companies and drives
- Creating a student and retrieving the one-time password
- Student login with that password
- Portal drive listing and drive detail
- Applying to a drive
- The offer appearing when the application is marked selected
- The ladder refusing a lower-paying drive, with its reason
- The offer disappearing when the decision is walked back
- Dashboard figures reflecting the overridden CTC
- Cascade deletion of students, drives and companies, including history and offer rows
- The origin guard: 403 for a foreign origin, pass for the configured origin and header-less clients

Client payload after code splitting

The charting library used to ride along with every route. Now it arrives on demand.

–55%

raw entry bundle; –56% compressed

404 kB

dashboard chunk (114 kB gzip), fetched once on first visit and cached by the browser

Every route other than the dashboard now paints from the smaller entry chunk, and the dashboard is loaded through a dynamic import with a suspense fallback.

Database queries per request

Three list endpoints were doing per-row work. Each now issues a constant number of queries.

Endpoint	Before	After
Drives list	$N + 2$	3
Applications list	$3N + 1$	1
Student detail	$\approx 2N + 4$	constant
Companies list	$N + 1$	2

How

- Joined eager loads for row data
- One grouped aggregate for counts
- Scalar column queries for dashboard figures

Why it matters on this deployment

The API runs on a serverless host against Neon, a serverless Postgres whose compute and storage are separated. Every request pays connection setup to a remote database, so removing dozens of round trips per page load is worth more than micro-optimising any single query. N is the number of rows returned.

LIMITATIONS

What this does not do yet

The gaps I would expect you to find, stated before you find them.



Unbounded lists

List endpoints return everything; real pagination is needed as a cohort approaches the importer's 5 000-row ceiling.



No login rate limiting

Tolerable for a campus tool behind institutional access; not sufficient for a public service.



UI logout leaves the cookie

Signing out clears the client state but does not yet call the logout endpoint, so the session cookie stays valid until it expires.



Tests are on demand

The integration suite is development-owned and not yet wired into continuous integration, so its guarantee depends on it being run.



Analytics are descriptive

Package averages use every offer record rather than one best offer per student, and the funnel is a current-state snapshot, not a round-by-round conversion.



Ladder trusts the advertised package

Until the cell overrides it, an inflated listing can temporarily admit students the rule would otherwise exclude.

Where the project goes next

Ordered by what the deployment actually needs first.

1

Server-side pagination

Keeps list endpoints predictable as cohorts grow

Next

2

Logout invalidates the session

Call the logout endpoint from the UI so signing out ends the cookie

Next

3

Login rate limiting

Closes the remaining security gap before any public exposure

Next

4

Resume upload

Attach a CV to the student record so the cell stops chasing email attachments

Then

5

Notifications from history

Drive the audit table into student notifications, so nobody misses a deadline

Then

6

Skill-based matching

Match student skill profiles against drive requirements to suggest drives

Later

What was built, and what it demonstrates

The system

- Two working portals over one role-guarded REST API
- Six core tables plus an offer table and an audit table
- Deployed on Vercel against Neon serverless Postgres

The contribution

- Eligibility concentrated in one server-side function with four call sites
- An offer ladder backed by automatically maintained placement records
- An append-only history that makes every decision replayable

The engineering

- A development integration suite of 23 REST-level assertions
- Entry payload cut from 740 kB to 336 kB (~56% compressed)
- Per-row database work replaced by joins and grouped aggregates

PlaceMate replaces the spreadsheet season with a system whose rules live in one place, whose offers are recorded as data, and whose decisions can be replayed from a log.

Thank you

Questions and demonstration

Tushar Kumar · Department of Computer Applications (BCA)
Maharaja Surajmal Institute, C-4 Janakpuri, New Delhi – 110058
Affiliated to Guru Gobind Singh Indraprastha University, Delhi

A live walkthrough of both portals is available on request.